程序设计综合实验

第 4 讲：链表的方方面面（链表第二讲：应用与复杂操作）

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-10-10

第 4 讲：链表的方方面面（链表第二讲：应用与复杂操作）

跨讲衔接：起点是第 3 讲的课下练习

跨讲衔接：起点是第 3 讲的课下练习

链表这一类结构一次课讲不完。第 3 讲把基础讲完：结构与内存、选型判据、单链表的基本操作、C 与 Julia 两侧的实现、以及 ASEP 的链表表示；课下留了四道题，让这套操作在手上过一遍。本讲**不再重复**第 3 讲的结构定义、内存代价、选型判据与基本操作，需要用到时只作回指，不再展开。

第 3 讲课下练习的回收与本讲范围

第 3 讲的课下练习共四题，三题动手、一题只提问。

题	要求	本讲用在哪里
题目一	用 C 与 Julia 各写一遍单链表的创建、插入、删除与打印	本讲每一题的实现都建立在这四个动作上；两侧的写法沿用第 3 讲的约定（C 侧带头节点与 `Node **`，Julia 侧抽象类型分派）
题目二	把 ASEP（Asymmetric Simple Exclusion Process，不对称简单排斥过程）的数组版格点与粒子表示改写成链表并跑通	本题是链表在一条真实计算任务上的落地，本讲不再讲 ASEP；写通过的链表示本讲练习的测试链
题目三	从存储结构、内存分配、访问时间的步骤、插入与删除的步骤四个方面比较链表与一维数组	本讲的每一张代价表都沿用这四条口径，只把对象从单链表换成双链表、循环链表与静态链表
题目四	思考题：是否可能把链表与一维数组的优点结合起来	本讲类型扩展一节的静态链表（选读）就是这道题的答案；本题在第 3 讲只提出问题，不给答案

练习过程中出现的问题集中在四处：头指针与尾指针在哪一步被更新（题目一与题目二的删尾）、边界情形有没有先处理（空链表、只有一个节点的链表、删首与删尾）、哨兵（哑节点）这一写法省掉了哪些特判（题目一里两种写法各自出现的分支数）、链表在哪一类操作上比数组便宜（题目三的四个方面）。四处问题在本讲的位置是明确的：第一处与第四处回到每一题的代价账上，第二处集中在复杂操作一节，第三处与类型扩展一节的双链表接上。

看一遍题目四留下的问题，就得到了本讲的主线。第 3 讲把链表与数组当作两种互相替代的表示，判据的写法是“选哪一个”；题目四问的却是能不能不选。静态链表给出的回答是：数据放数组、次序放下标，于是按序号访问不再走链，插入与删除不再搬移数据；这条改动修改了第 3 讲的选型判据。

本讲的范围与路线图：先接课下练习，再成组做复杂操作题，然后把讲过的四种排序搬到链表上重算代价，接着补类型变体，最后是工程组织、课外阅读与练习作业。

环节	讲什么	承接哪里
跨讲衔接（本节）	回收第 3 讲的课下练习、四类出现的问题、范围与路线图	接第 3 讲的课下四题
复杂操作	合并、三等分、交换、逆转、拆分、环检测、相交、倒数第 k 个	接第 3 讲基本操作；合并与拆分正是归并排序的两个基本操作
链表上的排序	冒泡、插入、归并（拆分加合并）、三路快速排序的拆分	把讲过的四种排序换到链表上，重点讲代价结构的变化
类型扩展与静态链表	双链表、循环链表、静态链表	回答第 3 讲题目四的思考题；接第 3 讲的选型判据
工程实战	C 项目组织、头文件与源文件成对、函数库、通用节点类型（选读）	把两讲的链表代码放进一个可提交、可复现的项目
课外阅读	阅读标准库、Lisp 与 Scheme 的渊源、两种语言的分工	与第 3 讲 Julia 侧抽象类型链表呼应
练习与作业	课上三道衔接题；课后练习四道；作业在下一节布置	作业是任务与步骤的链表实现与文件 IO 设计

本讲与第 2 讲（排序与分治）的关系也要说清：第 2 讲在**数组**上讲过插入、冒泡、快速三种排序，并从快速排序里带出分治思想；归并排序与分治的一般结构由第 3 讲的附录补齐。那部分内容在本讲只作引用，不重复搬运。本讲做的是同一批算法换一种表示之后重算代价：数组上最贵的那一步是把元素搬来搬去，链表上这一步变成改指针，但链表没有按序号直接取节点的能力，于是“找位置”与“分区”变贵。同一算法在两种表示上的代价结构对照表，放在排序一节。

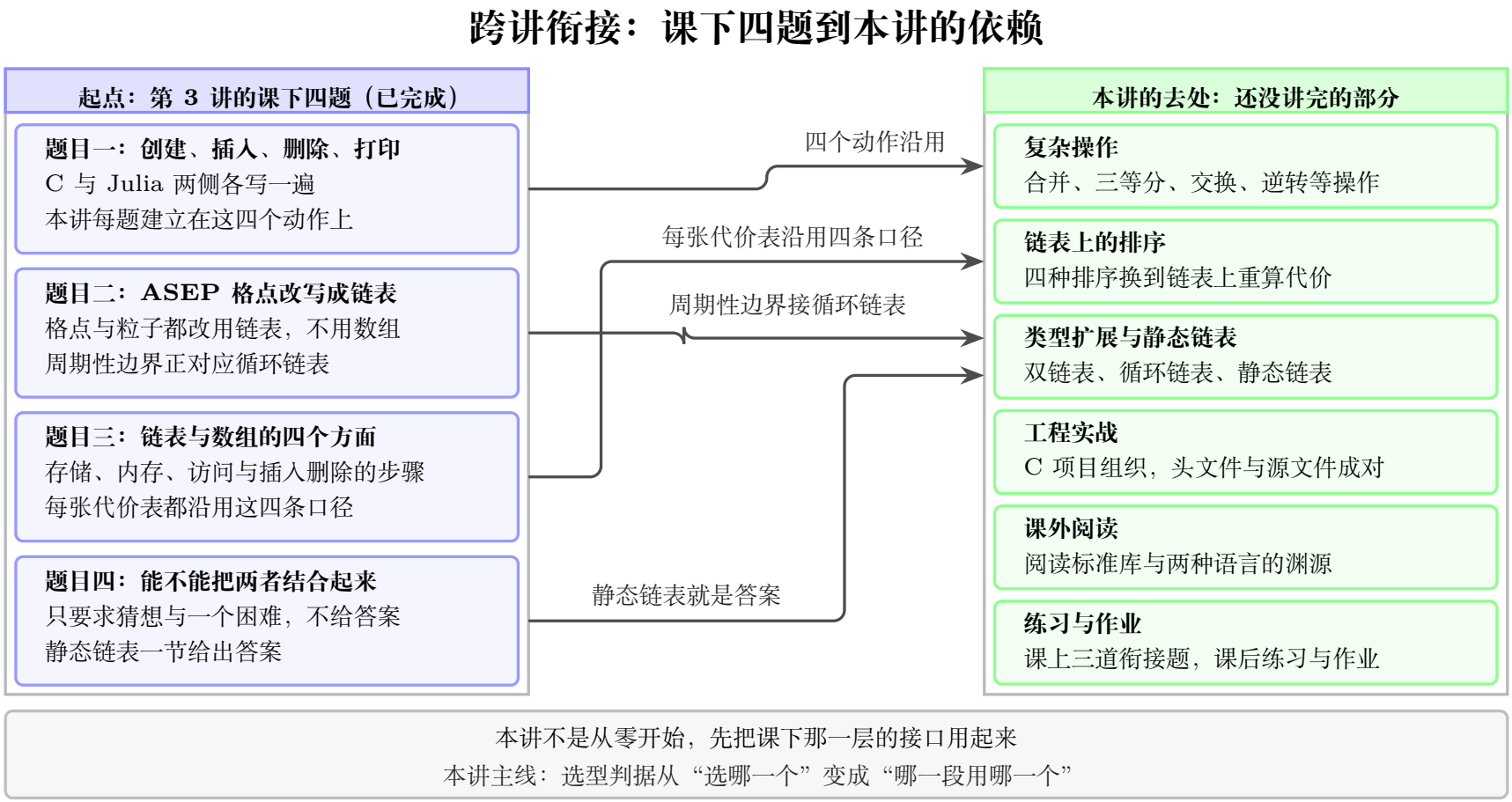


图 1: 课下四题到本讲各节的依赖

Q 1 AI 1 详解 1

链表上的复杂操作：合并、三等分、交换、逆转、检环、相交与倒数第 k 个

链表上的复杂操作：合并、三等分、交换、逆转、检环、相交与倒数第 k 个

第 3 讲讲过九道步骤题与两道课上小题，本讲把其中七道综合些的题成组做完：先做四道有现成实现的题（合并两条有序链、长链三等分、交换指定节点、逆转链表），把每题的操作步骤、边界情形与代价讲透；再做三道没有现成实现的题（环检测并找环起点、两链相交并找交点、倒数第 k 个节点），这三题要求给出**为什么正确**，而不只是给出代码。

本讲实测口径（全讲仅此一处声明）：C 侧用 clang 编译运行、Julia 侧用 julia 运行，两侧同一组用例，正文引用的每个数字都取自示例程序的原始输出。三个计数器的口径定义如下，本讲后面各节沿用：

- **走链次数**：指针沿 `next` 前进一步记一次（包括为了取下一个节点、以及为了把结果链的尾指针前移；前进一步也口语作“走一步”，正文里“走链 10 步”与表里的走链次数是同一个计数）。
- **改指针次数**：写入 `next` 指针、改变链的结构一次记一次。
- **比较次数**：两个节点的数据之间做过一次比较即记一次。

链统一取带头节点写法：**哑节点**（也叫哨兵节点）是链首一个不存有意义数据的节点，它存在的作用是让“在第一个数据节点之前插入”与“改第一个数据节点”这两件事在程序结构上变成同一件事。

合并两条有序链表与长链三等分：分与治的两个基本操作

合并（merge）指把两条已经各自有序的链合并成一条有序链：不新建数据节点，只把原有的节点按数据大小依次接在结果链的尾部。做法是三个指针：`p` 与 `q` 各指两条输入链的当前节点，`tail` 指结果链的末尾。

步骤：每轮比较 `p` 与 `q` 的数据，把较小的那一个接到 `tail` 之后，再把这个指针前进一步；当其中一条链走完时，停止逐节点比较，把**剩下的一整段**直接接在结果链尾部。最后一步是整段相接，不是逐个节点相接，这是本题第一个要点：走完的那条链已经有序，剩下的那一段也一定不小于已接上的全部节点。

实测（`./complex_ops`，两条各三个节点的链，`1 → 3 → 5` 与 `2 → 4 → 6`）：合并得到 `1 → 2 → 3 → 4 → 5 → 6`，走链 10 步，改指针 8 次，比较 5 次。10 步的构成值得数一遍：5 次从输入链取节点各前进一步，5 次把结果链的尾指针前移一步，所以**每接上一个节点要走两步**；5 次比较恰好是 $m + n - 1$，其中 $m = n = 3$，这也不是巧合：每一轮比较消耗一个节点，最后一次比较之后剩下的一整段不需要再比。

边界情形三条，都跑了：

情形	输入	输出	走链	改指针	比较
空链参与合并	空链与 `2 → 4 → 6`	`2 → 4 → 6`	0 步	3 次	0 次
单节点链与空链	`7` 与空链	`7`	0 步	3 次	0 次
两条等长有序链	`1 2 3 4` 与 `1 2 3 4`（含重复值）	`1 1 2 2 3 3 4 4`	14 步	10 次	7 次

空链参与合并时走链 0 步、比较 0 次，只有 3 次指针改写：一次把剩下一整段接到结果链尾，两次把两条输入链的哑节点与结果链脱开。第三行的重复值说明比较用的是“小于等于”，相等时优先取第一条链的节点，因此两条链的相对次序在相等处保持不变。

长链三等分（split into three）指把一条长链按结点数切成三条长度大致相等的短链。在两个节点之间置 `next = NULL` 的位置称为**断点**，三等分需要两个断点。做法是**快慢指针**（fast and slow pointers，两个指针同向走但每轮的步数不同，用步数比把某个位置算出来），分两趟各定一个断点：

1. **第一个断点**：快指针每轮最多前进三步、慢指针每轮前进一步。快指针走到链尾时，慢指针恰好停在第 $\lfloor n/3 \rfloor$ 个节点上，把它当作第一段的尾，在它和下一个节点之间切断。
2. **第二个断点**：在余下的那一段上，快指针每轮走两步、慢指针走一步，循环在快指针前方不足两个节点时停下；停下时慢指针恰好停在第 $\lfloor m/2 \rfloor$ 个节点上（m 是余下那一段的长度），在它和下一个节点之间切断。

两个断点的位置都由步数比算出来，因此**不需要先数一遍链长**，两趟走链合计 $O(n)$。取整规则用 $\lfloor \cdot \rfloor$：余数一律给前面那一段。

为什么用两趟双指针，而不是一趟三个指针：一趟三指针并非不可行，难在边界。$n \le 2$ 时链上的节点填不满三个非空的段，实现必须为这些退化情形单独写守卫；失败实现（见本节详解的实测对照）只防住了“一个数据节点也没有”这一种，$n = 1$、$n = 2$ 时把同一个节点接进了多条链。而且段边界与“先接哪一段、后接哪一段”的次序，要求三个指针在全程同时维护同一组不变式，出错面大。两趟方案每一趟只有一个不变式（快慢步数之比定出断点），第二个断点从余链的头部重新起算，$n = 0$、1、2 各分支的判定点彼此独立、可逐个实测

合并两条有序链表与长链三等分：分与治的两个基本操作

下面 n 从 0 到 10 逐个跑的实测表就是全分支覆盖。代价是多走一遍链：合计仍是 $O(n)$，常数翻倍，换来的是每个断点独立可验证。

实测（同一程序，n 从 0 到 10 逐个跑）：

n	三段长度	走链次数	改指针次数
0	0 / 0 / 0	0	1
1	1 / 0 / 0	2	1
2	1 / 1 / 0	3	4
3	1 / 1 / 1	4	4
4	2 / 1 / 1	6	4
5	2 / 2 / 1	10	4
6	2 / 2 / 2	11	4
7	3 / 2 / 2	13	4
8	3 / 3 / 2	17	4
9	3 / 3 / 3	18	4
10	4 / 3 / 3	20	4

表里两列边界要单独看。$n = 1$ 时三段长度是 1/0/0：第一段留下唯一那个节点，第二、三段是两条带头节点的空链，因此仍然可以照常按“带头节点的链”去用它们，不必在调用处再判断空指针。$n = 5$ 时按取整规则分成 2/2/1，$n = 7$ 分成 3/2/2：余数都给前面的段，理由是后面两段还要继续被切或被递归处理，让它们尽量短些更合算。改指针一列在 $n \geq 2$ 之后稳定为 4 次：两个断点各切断一次，再加两次把第二、三段接到各自新哑节点上；$n = 1$ 时没有可切的地方，只有一次。

这两题为什么放在一起讲：拆分长链与合并有序链正是链表上归并排序的两个基本操作，也就是第 3 讲附录里“分”与“治”的一般结构在链表上的对应物。这个对应关系在排序一节会完整接上。

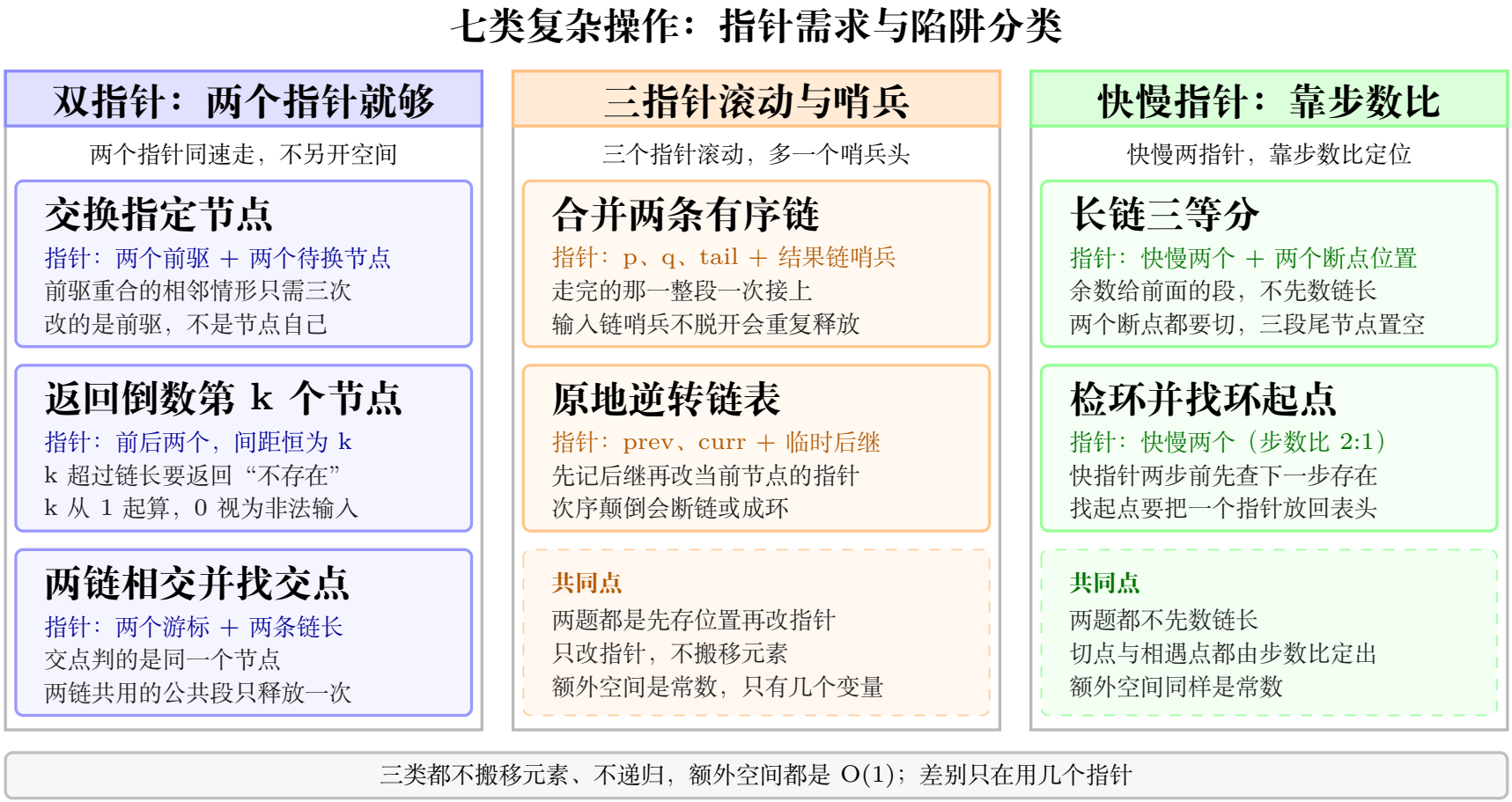


图 2: 复杂操作的指针需求与陷阱分类



图 3: 长链三等分：两个断点与快慢指针的步数比

Q 2 AI 2 Q 3 详解 2 详解 3 常见错误与要点（两题）。

1. **整段相接被写成逐个相接**：走完的那条链剩下的一整段有序，直接接上即可；逐节点相接会把走链步数从 $2(m+n-1)$ 抬到 $2(m+n)-2$ 剩余节点数。

2. **输入链的哑节点不脱开**：合并之后，两条输入链的哑节点仍指向被接走的节点，谁先释放谁就制造一次重复释放；C 侧的堆检查会直接报出 `free(): double free detected`。修法是合并完成后立刻把两个哑节点的 `next` 置空。
3. **三等分的分点取整只说“大致相等”**：$n = 4$ 时的 2/1/1 与 1/1/2 都叫“大致相等”，但两者的第二个断点位置不同，实现与验收要对齐到同一个规则（本节的规则是余数给前面的段 2/1/1）。
4. **三段尾节点的指针域不置空**：三段本来是连着的，切断必须落在两个断点上；漏掉一个断点，遍历会跨段走下去。
5. **只测“三段看起来对”**：$n = 1$ 与 $n = 2$ 要在用例里显式跑，并把三段的节点数加起来与 n 对照；段长之和大于 n 就是节点被两条链共用。
6. **定位指针错误的手段**：这类错误没有编译期检查，本讲口径是用调试器 lldb 单步跟一次指针的赋值，或者给遍历打印加上限、把链的形状一次打出来看。

交换指定节点与逆转链表：三指针的滚动与相邻情形的坑

交换两个指定节点指把链上第 i 个与第 j 个数据节点互换位置；位置从哑节点起算，第 0 个就是第一个数据节点。做法分两步：先找到这两个节点的**前驱**（指向某个节点的那个节点），再按“相邻”与“不相邻”两种情形分别接指针。

不相邻情形改写四次：`prevI` 的 `next` 改指 `nodeJ`，`prevJ` 的 `next` 改指 `nodeI`，再把两个节点各自的 `next` 互换。相邻情形只有三次，因为这时两个前驱关系重合：第 j 个节点的前驱就是第 i 个节点本身，于是“把 prevJ 的 next 改指 nodeI”这一步会先把节点指回自己，必须由后面的两步覆盖回来。实测（六节点链 `1 2 3 4 5 6`）：

情形	操作	结果	走链	改指针
不相邻	交换第 1 与第 4 个	`1 5 3 4 2 6`	3 步	4 次
相邻	再交换第 2 与第 3 个	`1 5 4 3 2 6`	2 步	3 次
首尾	再交换第 0 与第 5 个	`6 5 4 3 2 1`	4 步	4 次
同位置	交换第 3 与第 3 个	不变	0 步	0 次

相邻情形比不相邻少一次，这是“多一个前驱就少一次改写”的第一次出现：双链表的删已知节点一节还会再出现一次。另一种常见写法不区分相邻与不相邻，只按不相邻情形的四次改写去写，在六节点链上跑遍全部 30 组有序位置（$i \neq j$ 共 6×5 组），与期望结果**逐组一致**，原因是相邻时那三次赋值恰好互相抵消。

逆转链表（reverse in place）指把链上节点的次序整个倒过来，要求不新建数据节点。做法是三个指针滚动；三个指针分别是**前驱** `prev`（已经翻好的那一半的当前头）、**当前** `curr`（正在处理的节点）、以及处理前先记下的**后继** `next`：

1. 先把 `curr` 的后继记下来（这一步不做，后面改指针就会丢掉这条链的后半段）。
2. 把 `curr` 的 `next` 改指 `prev`，于是 `curr` 接进了已经翻好的那一半。
3. `prev` 前移到 `curr`，`curr` 前移到刚才记下的后继，进入下一轮。

4. 循环结束后把哑节点的 `next` 接回 `prev`，它现在是新的第一个数据节点。

为什么赋值次序不能颠倒：每一步只有一份“后继地址”，它存在 `curr->next` 里；先写 `curr->next = prev` 就会把这份唯一的拷贝覆盖掉，后半条链再也走不回去，结果是断链或（在交换相邻节点这类操作里）成环。实测（五节点链 `1 2 3 4 5`）：逆转得到 `5 4 3 2 1`，走链 5 步（每个节点被访问一次）、改指针 6 次（五个节点各一次，加上哑节点接回的一次）；再逆转一次回到原序，计数完全相同。

代价结构：两题都不搬移元素，只改指针。逆转是**原地操作**（in place，只改动链本来的节点与指针，不申请与元素个数同阶的额外空间），额外空间 $O(1)$，时间 $O(n)$；交换两题都要求先找到位置，找位置是走链，代价 $O(n)$，拿到位置之后的改指针是 $O(1)$。

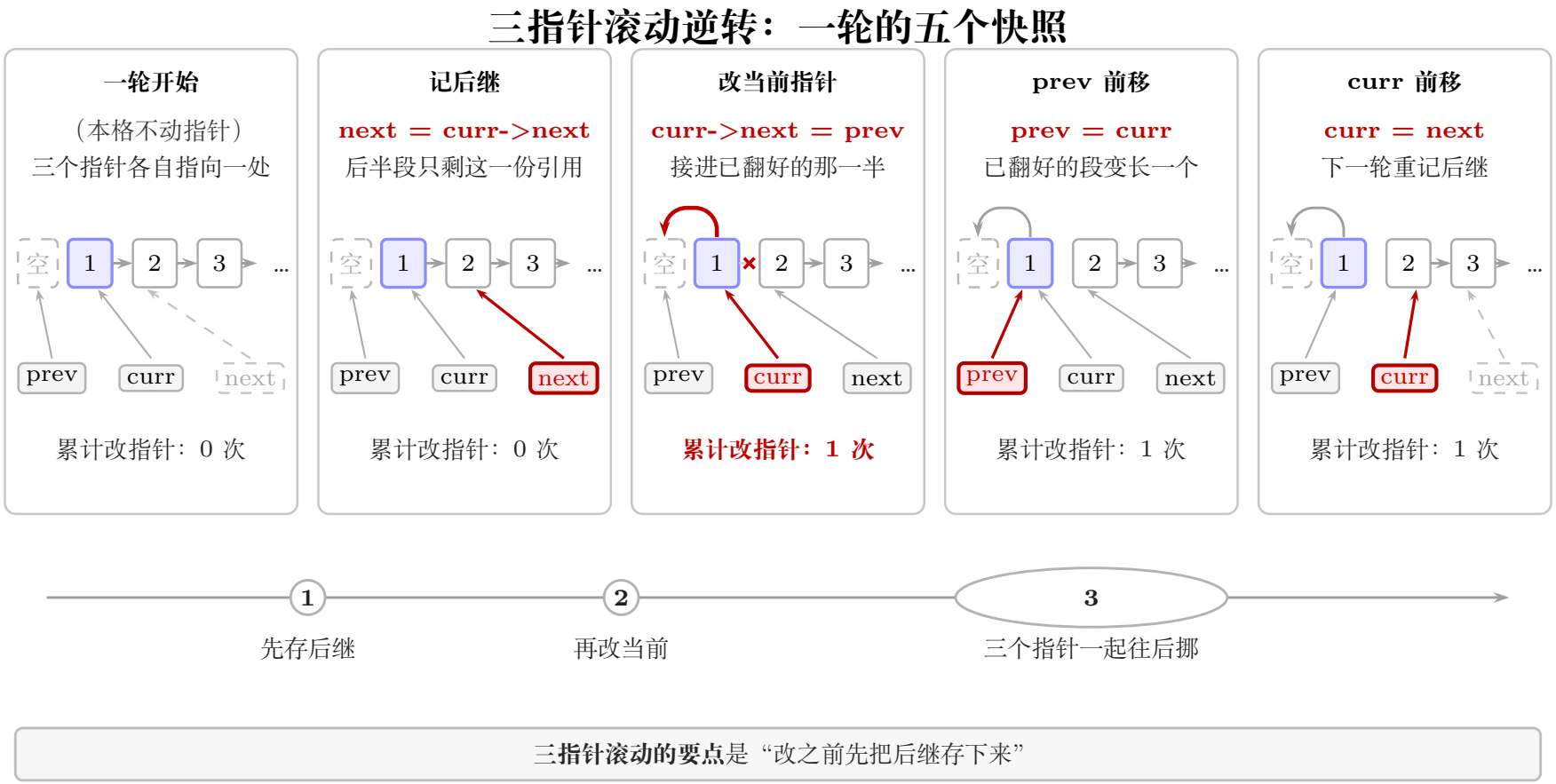


图 4: 三指针滚动逆转的一轮五格快照

Q 4 AI 3 详解 4 详解 5 常见错误与要点（两题）。

1. **逆转里先改 `curr->next` 再取后继**：这是本节最常出现的一处错，后果是后半条链从链上头再也走不到。

2. **交换时把相邻情形当不相邻写**：实跑可能仍然正确（赋值互相抵消），但推理链断了一环；要养成“先看两个前驱是否重合”的习惯。
3. **忘了交换要改的是前驱而不是节点自己**：`nodeI->next` 与 `nodeJ->next` 是互换的对象，`prevI->next` 与 `prevJ->next` 是重新挂接的对象，两组不能混。
4. **首节点参与交换或逆转时忘了哑节点**：改首节点等于改头节点指向谁，这一处与第 3 讲删首节点是同一条边界。
5. **用“跑一遍看着对”验收**：两题的检查方式是数计数与回读链：改完一次就打印一遍链，并给打印循环加上限（本节程序用 40 个节点的上限），成环会立刻现形。

环检测、相交与倒数第 k 个：快慢指针与双指针法

下面三题除了代码，还要求写出**为什么正确**。三题的公共工具是**双指针法**（two pointers，用两个指针以不同的速度或不同的起点在同一结构上移动，用两个指针之间的关系代替分类讨论）。

检环与找环起点。 判定一条链上是否有环，用快慢指针：慢指针每轮走一步、快指针每轮走两步；若快指针先走到链尾，则无环；若两者相遇，则有环。正确性看两条：

1. **有环必相遇**：一旦两个指针都进入环，快指针每轮比慢指针多走一步，两者之间的环上距离每轮减少 1，因此有限轮内必定减到 0。
2. **无环不相遇**：快指针的步数是慢指针的两倍，在无环链上快指针先到空值，循环在相遇判定之前结束。

找环起点用第二段：相遇之后，把一个指针放回表头，两个指针改为同速前进，再次相遇处就是环起点。论证用步数关系：设表头到环起点的长度为 μ、环长为 λ，相遇时慢指针走过的步数是 $\mu + k$（k 是环内偏移），快指针走过的是 $2(\mu + k)$，两者之差 $\mu + k$ 必是环长的整数倍。于是 $\mu + k \equiv 0 \pmod{\lambda}$，从表头出发走 μ 步与从相遇点出发走 μ 步，会同时落在环起点上。

实测（`./complex_ops`，六节点链，尾节点的 `next` 指回值为 3 的那个节点）：

测试链	结果	走链
带环（环起点在值 3 处）	环起点 = 值 3，环长 4	16 步
同一条链断环之后	无环	9 步
单节点自成环	环起点 = 值 9，环长 1	3 步

带环那一次 16 步由两段构成：第一段快慢指针走到相遇，第二段两个指针同速走到环起点；环长 4 与“表头到环起点 2 步”这个数字合起来满足上面那条同余关系（$\mu + k$ 是 4 的倍数）。单节点自成环是最短的环，环长 1，两个指针在第一步就相遇。

两链相交并找交点。 两条链相交指它们从某个节点起共用同一段节点，**交点**指第一条被共用的节点。这里的“相交”不是指两条链在画图时搭在一起，而是指同一个节点（同一块内存、同一个下标）被两条链同时指向。做法是先求出两条链的长度、让长的那条先走长度差那么多步，使两个指针到交点的距离相等，再同步前进，第一次相等处即交点。正确性来自两条：

1. **交点唯一**：单个节点只有一个 `next`，两条链一旦走到同一个节点，之后的路径完全一致，因此交点不可能有两个。
2. **对齐有效**：交点之后的公共段长度相同，所以两条链长度的差就是交点之前的长度差；先消掉这段差，两个指针就会同时到达交点。

实测：链 A 为 `1 2 3 7 8 9`、链 B 为 `4 5 7 8 9`（`7 8 9` 是共用段），得到交点 = 值 7，走链 16 步；把两条链换成互不相交的 `1 2 3` 与 `4 5 6`，得到不相交，走链 12 步。

第二种解法：把一条链的尾接到另一条链的第一个数据节点上，从第一条链的头出发走，若两条链相交，则这条大链上出现环，且**环起点就是交点**。实测同一条用例：环起点 = 值 7，走链 21 步，与第一种解法的结论一致。这一步的意义在于它把相交问题归到检环问题上，用的是同一套快慢指针；代价是它临时改动了原链，验完之后必须还原。

倒数第 k 个节点。 做法是两个指针从同一个起点出发，先让前指针走 k 步，再让两个指针同速前进；当前指针走到链尾（空值）时，后指针恰好指着倒数第 k 个节点。正确性来自一条**不变式**（invariant，在整个过程中始终成立的关系）：两个指针之间的间距恒为 k 个节点，前指针到尾时后指针距尾正好 k 个位置。

实测（六节点链，k 从 1 试到 7）：

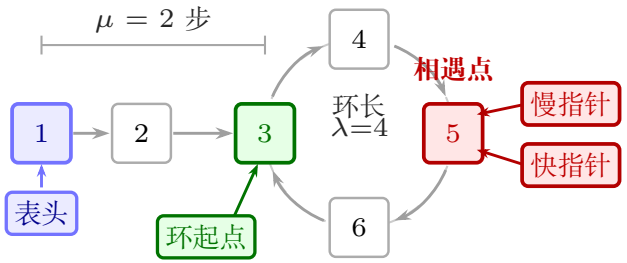
k	结果	走链
1	值 6	11 步
2	值 5	10 步
3	值 4	9 步
4	值 3	8 步
5	值 2	7 步
6	值 1	6 步
7	不存在（k 超过链长）	6 步

走链步数满足 $2n - k$：前指针走 n 步到尾（其中前 k 步是单独走的），后指针走 $n - k$ 步，合计 $n + (n - k)$。k 超过链长时前指针在走前 k 步的中途就碰到空值，直接返回“不存在”，不再走第二步。

三题的代价都是 $O(n)$ 时间、$O(1)$ 额外空间；空间能压到常数，原因是它们都只用了两个指针与几个临时变量，没有开数组，也没有递归。

快慢指针：相遇与找环起点的两段推演

第一段：两个指针从表头出发，快指针每轮两步、慢指针每轮一步



相遇时的步数账：两个指针各自的位移

表头到环起点的长度 $\mu = 2$ 步，环长 $\lambda = 4$ 步

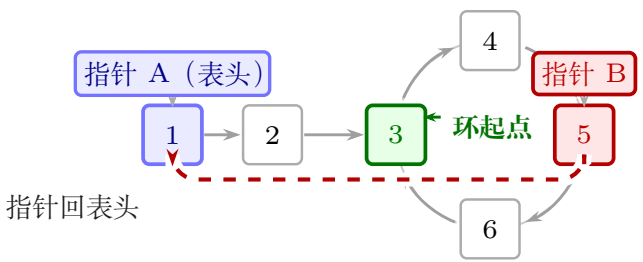
慢指针走过 4 步，记为 $\mu + k$

快指针走过 8 步，记为 $2(\mu + k)$

两者之差 4 步，恰是环长 λ 的 1 倍

两个指针最后落在同一个节点，所以差必是环长的整数倍

第二段：把一个指针放回表头，两者同速前进，再次相遇处即环起点



第二段：一个指针回表头，两者同速前进

指针 A 回到表头（值 1）重新出发

指针 B 留在相遇点（值 5）

两者各走 $\mu = 2$ 步，同时到达值 3

再次相遇处就是环起点

判环容易：快慢指针相遇就说明有环。

找起点要多一段推理：相遇后把一个指针放回表头，两者同速前进，再次相遇处才是环起点。

图 5: 快慢指针相遇与找环起点的两段推演



图 6: 两链相交的三种情形与判据

Q 5 **AI 4** **详解 6** **详解 7** **常见错误与要点（三题）。**

1. **判环的快指针不检查下一步是否存在**：快指针走两步之前必须先看 `fast ≠ NULL && fast→next ≠ NULL`，否则在偶数节点链的尾部会解引用空值。这是三题里最容易出现的崩溃点。

2. **找环起点时忘了把慢指针放回表头**：两个指针都留在相遇点再同速走，会绕环走一圈回到相遇点，得到的是相遇点而不是环起点。
3. **把“相交”理解成值相等**：两条链上出现相同的值不等于相交；交点判的是节点本身（C 侧是指针相等，Julia 侧是同一个对象）。
4. **相交题不先对齐长度**：直接同步前进会在长的那条链上错过交点。
5. **倒数第 k 个把 k 当 0 起的下标**：$k = 1$ 指尾节点，$k = n$ 指第一个数据节点；$k = 0$ 应当视为非法输入。
6. **释放共享节点两次**：相交用例里公共段被两条链同时指向，释放时只能释放一次；C 侧的堆检查会把这一处报成 `free(): double free detected`，Julia 侧由垃圾回收兜住，因此这类错误只在 C 侧暴露。修法是先断开其中一条链的接点再逐个释放。
7. **转成判环的解法忘记还原**：解法二临时改动了原链，验完必须把尾节点的 `next` 还原，否则后续用例会看到一条不该存在的环。

链表上的排序：与分治的衔接

链表上的排序：与分治的衔接

第 2 讲在**数组**这一种表示上讲过插入、冒泡、快速三种排序，并从快速排序里带出分治思想；归并排序，以及分治的一般结构（三条前提、一条终止要求与两个可调参数），由第 3 讲的附录补齐。本节不重新推导这四种算法，做的是另一件事：把它们换到链表上实现一遍，逐项重算代价。要看的结果是一个统一的结论，即同一算法的**代价结构**变了，而比较次数没变。

代价结构指同一算法在一种表示上，各步动作分别落在哪一类操作上、各出现多少次。本节把动作分成四类：**元素搬移**（把键值写进某个位置）、**改指针**（写 `next`，改变链的形状）、**走链**（沿 `next` 前进一次，用于定位）、**比较**（键值之间比较一次）。数组上只有搬移与比较两类，链表上多出改指针与走链两类，这正是本节所有数字的来处。

本节实测与复杂操作一节用同一份输入（固定随机数种子生成的 0 至 $n/2$ 之间的整数值，含重复值），程序在 `l04-linked-list-ops/examples/list_sort.c` 与 `list_sort.jl`；同一规模下两种语言的每个计数器逐个相同。正文引用的数字都取自这两份程序的原始输出。

链表上的冒泡、插入、归并与三路快速排序

冒泡排序在链表上有两种写法。**交换节点**（改指针）是链表本来的做法：比较相邻两个节点的数据，逆序时把这两个节点整体换位，改的是三条指针。**交换数据域**（搬元素）是把两个节点的数据互换，节点本身不动，改的是两个位置上的值。

实测（$n = 64$）：交换节点写法比较 2679 次、改指针 3126 次；交换数据域写法比较 3276 次、搬移 3126 次。三处数字值得逐个读：

- 两个 3126 是同一个数，它等于交换次数乘以三（$1042 \times 3 = 3126$）：交换一次要么写三条指针，要么写三个位置的数据。**交换次数一样**，差别只在写的是什么。
- 比较次数不同（2679 与 3276），原因是交换节点的写法在每次交换之后本轮会跳过一个比较对（被交换过去的那个节点要到下一轮才与它的新邻居比较），交换数据域的写法不跳。两种写法都是同一种算法的合法落法，比较次数的差别是写法造成的，不是算法造成的。
- 两行的代价合计是同一个数：交换节点写法把 3126 次全记在改指针一列，交换数据域写法把同样的 3126 次全记在搬移一列：交换节点那一侧的元素确实一步没动，代价只是换了名字。

插入排序在链表上的做法是：维护一条已经排好序的前缀，逐个把后面的节点插进前缀里的正确位置；找位置只能沿前缀走链。实测 $n = 64$ 比较 896 次、走链 897 步、改指针 126 次；$n = 256$ 比较 16563 次、走链 16569 步、改指针 508 次。这里最要紧的是**走链步数与比较次数几乎相等**：链表上找位置就是走链，找一次比较一次，所以插入排序在链表上省掉了搬移（改指针只有 508 次），却没省掉找位置的代价，最坏情形仍是 $O(n^2)$。这与第 2 讲在数组上实测的“插入排序在逆序输入上退化成平方级”是同一个复杂度，代价的构成却完全不同。

归并排序在链表上最自然，因为它由**两个基本操作**拼成：合并有序链（复杂操作一节的题一）与拆分长链（同节题二）。算法结构就是第 3 讲附录里“分、基线、治”三段：

1. **分**：把链从中间切开，取中点用快慢指针（快指针走两步、慢指针走一步），走链代价 $O(n)$。
2. **基线**：只剩一个节点的链天然有序，不需要排，直接返回。
3. **治**：把两条已排好的链合并成一条有序链，改指针代价 $O(n)$。

实测 $n = 64$ 比较 310 次、改指针 437 次、走链 1325 步、递归最大深度 6；$n = 256$ 比较 1722 次、改指针 2233 次、走链 7285 步、递归最大深度 8。两个深度（6 与 8）恰好是 $\log_2 64$ 与 $\log_2 256$：链表归并的递归深度只由分解形状决定，不随数据分布变化。

三路快速排序在链表上的做法是只做“拆分”这一半：取链头的数据作**枢轴**（pivot，用来把元素分成小于、等于、大于三类的参照值），把全部节点分到 `low`、`equal`、`high` 三条链上，再递归 `low` 与 `high`，最后把三条链接回去。实测 $n = 64$ 比较 306 次、改指针 334 次、走链 306 步、递归最大深度 9；$n = 256$ 比较 2129 次、改指针 2235 次、走链 2129 步、递归最大深度 15。走链步数与比较次数逐个相等（306 与 306、2129 与 2129），因为分区对每个节点只做两件事：与枢轴比一次、往下一步。两个深度（9 与 15）都大于同一规模上归并的深度（6 与 8）。

把四种排序在 $n = 64$ 与 $n = 256$ 上的实测摆在一起：

写法（链表侧）	$n = 64$ 比较	$n = 64$ 改指针	$n = 64$ 走链	$n = 64$ 深度	$n = 256$ 比较	$n = 256$ 改指针	$n = 256$ 走链	$n = 256$ 深度
冒泡（交换节点）	2679	3126	2679	0	55603	47397	55603	0
冒泡（交换数据域）	3276	0（搬移 3126）	3276	0	64260	0（搬移 47397）	64260	0
插入（找插入位）	896	126	897	0	16563	508	16569	0

写法（链表侧）	$n = 64$ 比较	$n = 64$ 改指针	$n = 64$ 走链	$n = 64$ 深度	$n = 256$ 比较	$n = 256$ 改指针	$n = 256$ 走链	$n = 256$ 深度
归并（递归版）	310	437	1325	6	1722	2233	7285	8
归并（迭代版）	310	438	1015	0	1722	2234	5563	0
三路快速排序（拆分）	306	334	306	9	2129	2235	2129	15

表里最要紧的三行是插入、归并与三路快速排序：插入的走链与比较同阶上升（$O(n^2)$），归并与三路快速排序的比较同阶（$O(n \log n)$ 量级），而**归并的递归深度固定、三路快速排序的深度随数据变**。

为什么链表上归并比快速排序更自然，这张表给了三条可以直接读出来的理由：

1. **比较次数随 *n* 涨得更快**：归并的比较次数从 310（$n = 64$）到 1722（$n = 256$），三路快速排序从 306 到 2129；$n = 64$ 时两者几乎相同，$n = 256$ 时三路快速排序多出 407 次。原因是分区在每一层都要让该层的全部节点与枢轴比较一次，而分区不一定均匀。
2. **枢轴取的是链头，分区容易不均**：链表上没有按序号取“中间那个元素”的能力，取枢轴要么扫一遍找，要么用链头；深度因此升到 15（$n = 256$，而 $\log_2 256 = 8$）。归并的深度固定为 $\log_2 n$，不靠枢轴运气。
3. **数组上快排的两条优势在链表上都不存在**：数组快排靠原地分区与连续存储的缓存友好，链表既不能原地分区成两段（要另接三条链），也没有连续存储可借。

第 2 讲在数组上的对比实测留下的印象是：快速排序在三种输入上都远快于冒泡与插入排序。这一印象在链表上被这三条改写成了“归并排序在链表上最自然”。这不是结论翻转，而是代价结构换了表示之后重算的结果。

分治结构在链表归并上的落地。 附录一给了分治的三条前提、一条终止要求与两个可调参数，它们在链表归并上逐条落地：子问题与原问题同型（同一条链的左右两半）；子问题相互独立（两条半链互不重叠、不互相依赖）；子问题的解可以合并（合并有序链，一次合一段）；分解有基线（只有一个节点的链）；参数一是每次分两份，参数二是基线规模 1。附录三给了递归与循环的适用判据，在链表上可以直接量出来：递归版的递归最大深度恰好是 $\log_2 n$（$n = 64$ 时 6 层、$n = 256$ 时 8 层），因此额外空间是 $O(\log n)$；迭代版（自底向上，先数长度，再按 1、2、4 的步长两两合并）递归深度 0，额外空间 $O(1)$。两版在同一个输入上的比较次数完全相同（310 与 1722），走链步数不同：$n = 64$ 时递归版 1325 步、迭代版 1015 步；$n = 256$ 时递归版 7285 步、迭代版 5563 步。走链步数差在**分解动作**上：递归版每层要用快慢指针走一遍找中点，迭代版一次数长度之后按步长直接分段。所以两版的选择判据是：能接受 $O(\log n)$ 的栈空间就写递归版（结构短、不易写错），要求额外空间为常数就写迭代版（分解的走链还更少）。

数组版与链表版归并排序：代价结构对照

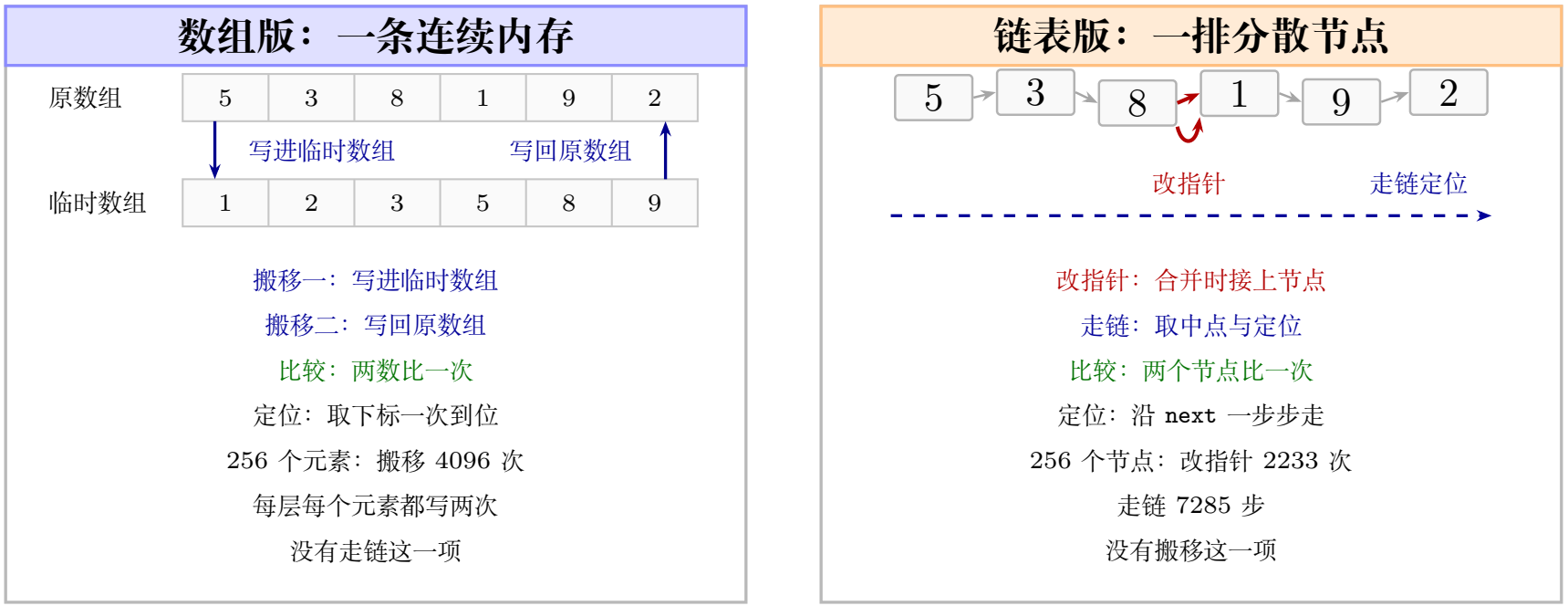


图 7：数组版与链表版归并的代价结构对照

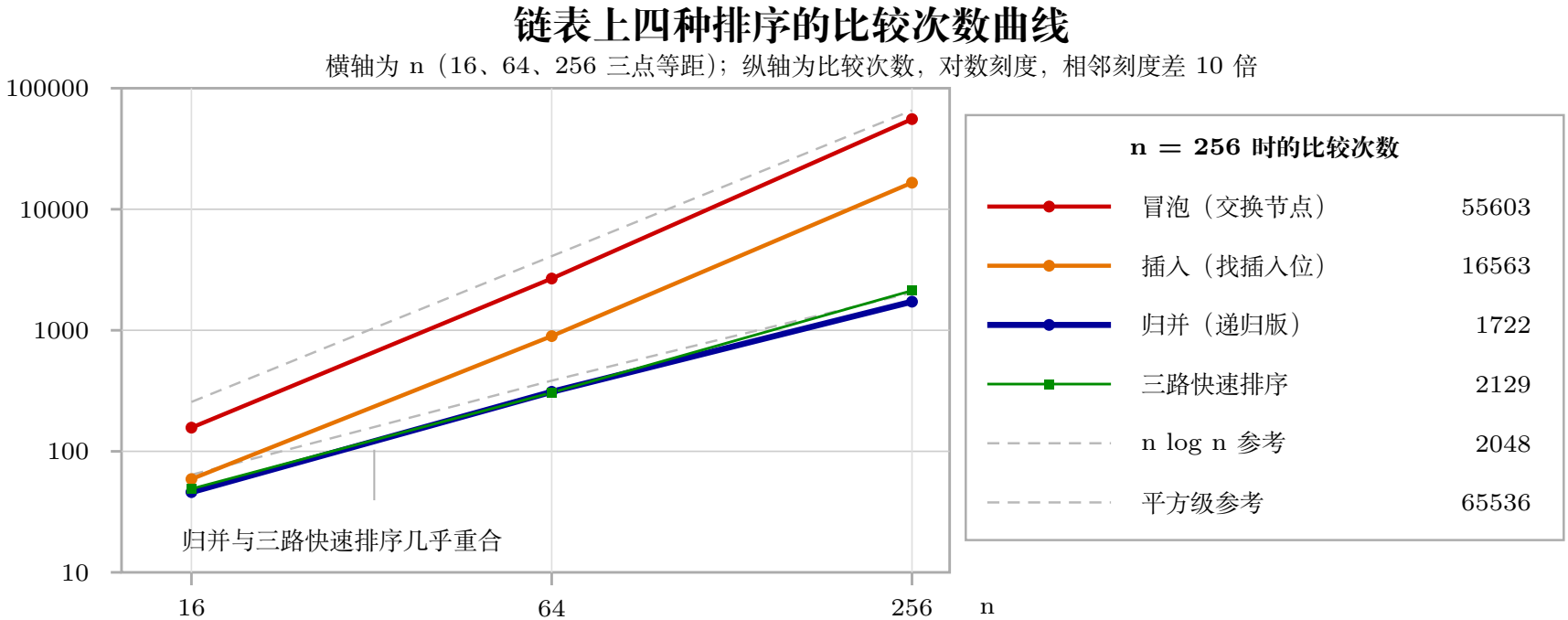


图 8: 链表上四种排序的比较次数曲线

Q 6 AI 5 详解 8 详解 9 常见错误与要点（本节）。

1. **在链表上照抄数组的实现**：数组靠下标定位，链表只能走链；照抄的后果通常不是错，而是代价从 $O(n \log n)$ 变成 $O(n^2)$（例如把“取中点”写成对链表按序号访问）。

2. **归并里合并时新建节点**：合并应复用原有节点，只改指针；新建节点的写法要多申请一份与元素个数同阶的内存，抵消了链表的优势。
3. **拆分时忘了把前半条链截断**：不截断就会在合并之后留下两条链共用一个后缀，链的形状被破坏且释放时会重复释放。
4. **递归深度当成常数**：链表归并的深度是 $\log_2 n$（$n = 256$ 时 8 层），输入规模大时这一层是真实的内存开销；要求额外空间为常数就换迭代版。
5. **枢轴取链头却不考虑重复值**：全部键相等时，二路分区的快速排序会退化成 $O(n^2)$ 深度；三路分区的 `equal` 链正是为这一类输入准备的。
6. **只用"结果有序"验收**：结果有序只能证明排序对了，不能证明元素没丢；验收要同时检查链上节点数与输入一致（本节程序的两项检查：链上值非递减、与输入多重集相同）。

链表的类型扩展与静态链表

链表的类型扩展与静态链表

前面各节讲的都是**单链表**：每个节点一个指针域、只能沿一个方向走。本节先把链表的常见类型补齐（双链表、循环链表），再把本讲的重点从“定义”转到“代价与选择”上：什么时候值得为每个节点多花一个指针，什么时候不值得。静态链表一节回答第 3 讲课下练习里的那道思考题（能否把链表与一维数组的优点结合起来），给出静态链表这一种做法、它的适用面与代价，第 3 讲的选型判据也在这一节收口。

程序在 `l04-linked-list-ops/examples/variants.c` 与 `variants.jl`、`static_list.c` 与 `static_list.jl`。计数口径沿用：**走链步数**是沿 `next` 或 `prev` 前进一步（两个方向都算）的次数，走到空值或哑节点的那一步也计入；**改指针次数**是写入 `next` 或 `prev` 的次数，**改下标次数**是写入下标数组或下标变量的次数。

双链表与循环链表：多一个指针值不值

三种常见类型的定义如下。

单链表（singly linked list）：每个节点由数据域与一个指针域组成，指针域指向后继节点，只能沿一个方向走；尾节点的指针域为空值（C 侧 `NULL`，Julia 侧 `nothing`）。

双链表（doubly linked list）：每个节点由数据域与两个指针域组成，一个指向后继节点、一个指向**前驱**节点（前驱指在链上排在它前面的那个节点），因此可以朝两个方向走。它换来的是“删除一个已知节点时不必再找前驱”，代价是每个节点多一个指针的内存，以及插入与删除时要**成对**维护两条指针（任一节点的 `next` 改了，被指向的那个节点的 `prev` 也要跟着改）。

循环链表（circular linked list）：让单链表的尾节点的指针域指回头节点（首尾相接，环由数据节点组成），于是任意节点都可以当作起点，从任何一个节点出发沿着指针走都能回到原处。它适用的场景是**环形遍历**（走一圈回到起点，判终止用计数而不是判空值）、**周期性边界**（走到一端就从另一端出现，第 3 讲 ASEP 练习里的边界正是这一条）、以及约瑟夫问题这类围成圈逐个出列的过程。

C 侧的双链表节点定义与建链（`examples/variants.c`）：

```c
typedef struct DNode {          // 双链表节点：两个指针域
    int data;
    struct DNode *next;
    struct DNode *prev;
} DNode;

typedef struct DList {          // 双链表结构：头指针与尾指针
    DNode *head;                // 哑节点
```

```
    DNode *tail;                // 尾节点；空链时与 head 同时指向哑节点
} DList;
```

```
DList dBuild(const int *a, int n) {
    DList l;
    l.head = dNew(0);           // 哑节点
    l.tail = l.head;            // 空链时尾指针与头同时指向哑节点
    for (int i = 0; i < n; i++) {
        DNode *p = dNew(a[i]);
        p->prev = l.tail;       // 先接前驱
        l.tail->next = p;       // 再接后继（成对的两条指针）
        l.tail = p;             // 尾指针后移
    }
    return l;
}
```

Julia 侧同两个类型与同一段建链（examples/variants.jl）。Julia 没有指针，两个指针域是两条对象引用，空值用 nothing：

```
mutable struct DNode            # 双链表节点：两个指针域
    data::Int
    next::Union{DNode,Nothing}
    prev::Union{DNode,Nothing}
end

mutable struct DList            # 双链表结构：头指针与尾指针
    head::DNode
```

```
    tail::DNode
end

function d_build(a::Vector{Int})
    head = d_new(0)              # 哑节点
    l = DList(head, head)        # 空链时尾指针与头同时指向哑节点
    for x in a
        p = d_new(x)
        p.prev = l.tail          # 先接前驱
        l.tail.next = p          # 再接后继
        l.tail = p
    end
    return l
end
```

多一个指针值不值，要看的是操作代价与内存代价两侧。实测（六节点链 `1 2 3 4 5 6`，两语言在每个计数上逐个相同）：

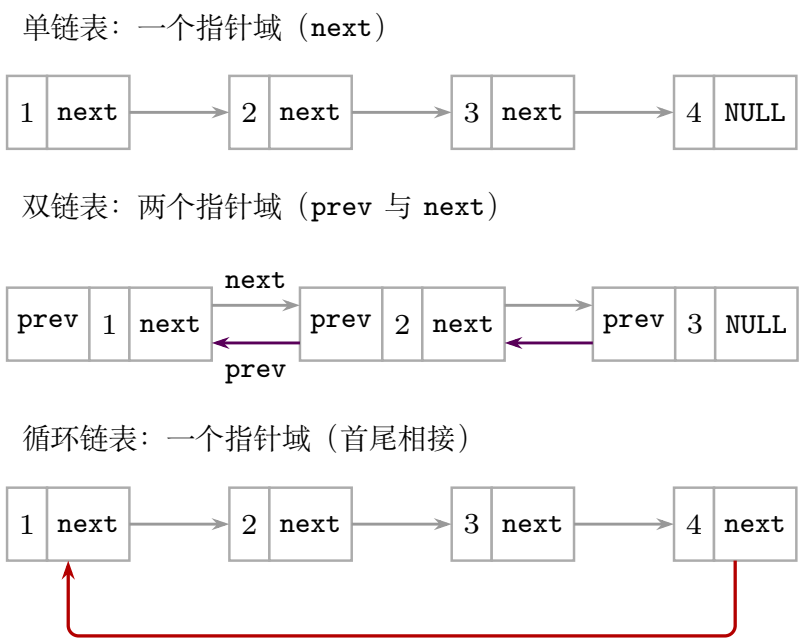
操作	单链表	双链表
删除一个已持有指针的节点（第 4 个）	走链 3 步、改指针 1 次	走链 0 步、改指针 2 次
删除尾节点（单链表只有头指针；双链表维护尾指针）	走链 5 步、改指针 1 次	走链 0 步、改指针 2 次
从尾向前遍历整条链	不支持；取尾节点的前驱要重走 5 步	走链 6 步
节点内存（C 侧 `sizeof`）	16 字节	24 字节
六个数据节点加一个哑节点（C 侧）	112 字节	168 字节
节点内存（Julia 侧 `sizeof`）	16 字节	24 字节

三行操作代价与三行内存代价合起来就是这道题的完整答案。**删除一个已经持有指针的节点**：单链表必须先找到它的前驱，而找前驱只能从表头走一遍，所以走链步数等于这个节点前面有多少个节点（第 4 个节点要走 3 步）；双链表直接读 `target→prev`，走链 0 步，改两条指针。**删除尾节点**：单链表即使维护了头指针，也要走一遍找新的尾节点（六节点链走 5 步），双链表用尾指针往回退一格就是新尾节点，走链 0 步。**反向遍历**：单链表根本做不到，取尾节点的前驱要重新从表头走 5 步；双链表沿 `prev` 走 6 步把整条链倒着打印出来。**内存**：C 侧 `sizeof` 量到 16 字节与 24 字节，六个数据节点加一个哑节点是 112 字节与 168 字节，即多出一个指针域、整条链多 50% 的内存；Julia 侧 `sizeof` 给出同样的两个数（Julia 的 `Int` 占 8 字节，C 的 `int` 占 4 字节但要补 4 字节对齐，两侧在这个规模上恰好相同）。

循环链表的操作代价实测：把六节点链的尾节点 `next` 指回第一个数据节点之后，从尾节点走 1 步到值 1、走 6 步回到出发的那个尾节点（值 6）。**约瑟夫问题**（n 个人围成一圈，从第 1 个人开始报数，报到 k 的人出列，再从下一个人继续）在 $n = 7$、$k = 3$ 上的出列顺序与走链、改指针计数见本节详解。这个问题的实现要用到循环链表的两个特点：终止靠计数（每出列一个人把计数减一，不看空值），出列靠前驱跨过（循环链表里“前驱”要在走链过程中自己维护，因为单链表的节点没有 `prev`）。

选择判据三条：一、频繁删除或插入、且已经持有目标节点时，双链表值得（走链 3 步对 0 步、5 步对 0 步）；二、需要双向遍历或需要经常访问前驱时，双链表值得；三、内存受限、节点数量很大、或者只需要单向扫描时，不值得（每个节点多一个指针、整条链多 50%）。第三条与第 3 讲“链表不是更快，而是在另一类操作上更便宜”是同一种提醒：多出来的指针要一直付内存钱，换来的只是若干类操作上的走链步数。

三种链表类型对照



尾节点 next 指回第一个数据节点

节点内存	能否回头走	删已知节点	尾节点判定
16 字节	不能	走链 3 步	后继为空值
24 字节	能，沿 prev	走链 0 步	后继为空值
16 字节	不能	走链 3 步	无空值，靠计数

代价数字取自本节六节点链实测（删除第 4 个节点）

多一个指针域换来能回头走与更少的走链步数，
代价是每个节点多占一份内存、插入与删除时多改一条指针。

图 9: 三种链表的结构与操作代价对照

Q 7 AI 6 详解 10 常见错误与要点（双链表与循环链表）。

1. **双链表插入时两条指针没有成对维护**：新节点的 prev 与后继节点的 prev 都要改，漏一条会让反向遍历在某一处跳到错的节点上；这类错误不会破坏正向遍历，因此靠正向打印发现不了。

2. **删尾之后忘记更新尾指针**：尾指针仍指着已释放的节点，下一次尾插会写进已经还给系统的内存。
3. **哑节点的 `prev` 没定义**：带头节点的双链表里，第一个数据节点的 `prev` 应当是哑节点还是空值，要在项目里先定下来，两种约定混用会在删首节点时出错。
4. **循环链表的终止还写成判空值**：圈上永远没有空值，遍历会一直转下去；循环链表的遍历都用计数控制。
5. **循环链表释放时不断环**：释放函数按“走到空值为止”写会死循环；先把尾节点的 `next` 置回空值再释放，或者按计数释放。
6. **把“双链表更高效”当成普遍结论**：它的高效只落在删除与插入这两类操作上，且以每个节点多一个指针域为代价；只读扫描的场景用双链表是净亏。

静态链表（选读）：用数组兼顾两者的优点

第 3 讲课下练习的第四题问的是：是否可能把链表与一维数组的优点结合起来。当时的答案只要求给出猜想与一处困难。

先把两边的优点列清楚。数组的优点来自连续存储：按序号访问是一次地址运算（$O(1)$），没有额外的结构开销。链表的优点来自指针：插入与删除只改指针、不搬元素，重排或排序时不必移动数据本身。两者的困难也清楚：数组一次下标运算就能定位，可一旦要重排就得搬元素，元素越大搬得越贵；链表不搬元素，可定位只能走链。

静态链表（static linked list，也叫游标链表 cursor-based linked list）把两者的优点摆在一起：**数据放进一个数组**（内存一次分配完），**次序放进一个下标数组**，用下标代替指针。落地写法是两个数组加一条空闲链：

- **数据数组** `data[i]`：存第 i 个槽位的数据。
- **下标数组** `next[i]`：存第 i 个槽位的后继槽位下标，-1 表示没有后继。
- **空闲链**：所有空闲槽位用下标串成一条链，`free` 是它的头。申请一个槽位就是取空闲链的头，释放一个槽位就是把它挂回空闲链。

这套写法的关键是：**链的形状完全由下标数组给出**，与数据在数组里的物理次序无关。重排、排序、按新次序遍历都只改下标数组，数据数组一个位置都不动。

若把结构写成三个数组（数据、下一个、位置），后两个数组的职责会重叠；按“下标模拟指针”的实际做法，两个数组加一条空闲链就够了，第三个数组在这里是冗余的。

实测（`./static_list`，八个槽位，槽位 0 是哑节点，尾插 `5 3 1 4 2 6`；Julia 侧下标从 1 起，槽位编号整体加一，哑槽位是 1）：

阶段	计数	两个数组的内容（数据数组 / 下标数组）
初始化	改下标 10 次	数据全 0；下标数组把空闲槽位串成一条链
尾插 6 个元素	每个元素 4 次，共 24 次；数据写入 6 次	数据 `0 5 3 1 4 2 6 0` / 下标 `1 2 3 4 5 6 -1 -1`（还剩槽位 7 空闲）
删除值 3	走链 1 步、改下标 3 次	数据不变 / 下标 `1 3 7 4 5 6 -1 -1`，槽位 2 挂回空闲链（空闲链 2 → 7）
重排为升序	走链 5 步、改下标 15 次、**数据搬移 0 次**、比较 7 次	数据 `0 5 3 1 4 2 6 0`（一个位置没动）/ 下标 `3 6 7 5 1 4 -1 -1`
再尾插 7、8	各 4 次	空出来的槽位 2 被重新用上：数据 `0 5 7 1 4 2 6 8`，下标 `3 6 7 5 1 4 2 -1`
再尾插 9（第 8 个元素）	失败	空闲链已空，容量固定为 8 个槽位

表里最要紧的两行是“重排”与“尾插 4 次”。重排那一行：逻辑次序从 `5 1 4 2 6` 变成 `1 2 4 5 6`，下标数组从 `1 3 7 4 5 6 -1 -1` 变成 `3 6 7 5 1 4 -1 -1`（空闲链那一格 `2 → 7` 未受影响），而数据数组前后**完全相同**。同一组数据、同一个次序，用数组版插入排序重排一次作为对照：数据搬移 8 次、比较 7 次。两次比较次数相同（7 次，比较由算法决定），搬移这一列的差别就是这道思考题的答案：静态链表把 8 次数据搬移换成了 15 次下标改写。

写下这份对照时要带上单位，否则会得出相反结论。若一个元素是一条 64 字节的记录、一个下标是 4 字节，数组版这次重排要搬 $8 \times 64 = 512$ 字节，静态链表的 15 次下标改写是 $15 \times 4 = 60$ 字节，相差约 8.5 倍。元素越大，这份对照越偏向静态链表；元素是一个整数时，两边都是十几个字节，静态链表并不占便宜。这正是“重排大对象”这一条适用面的量化版本。

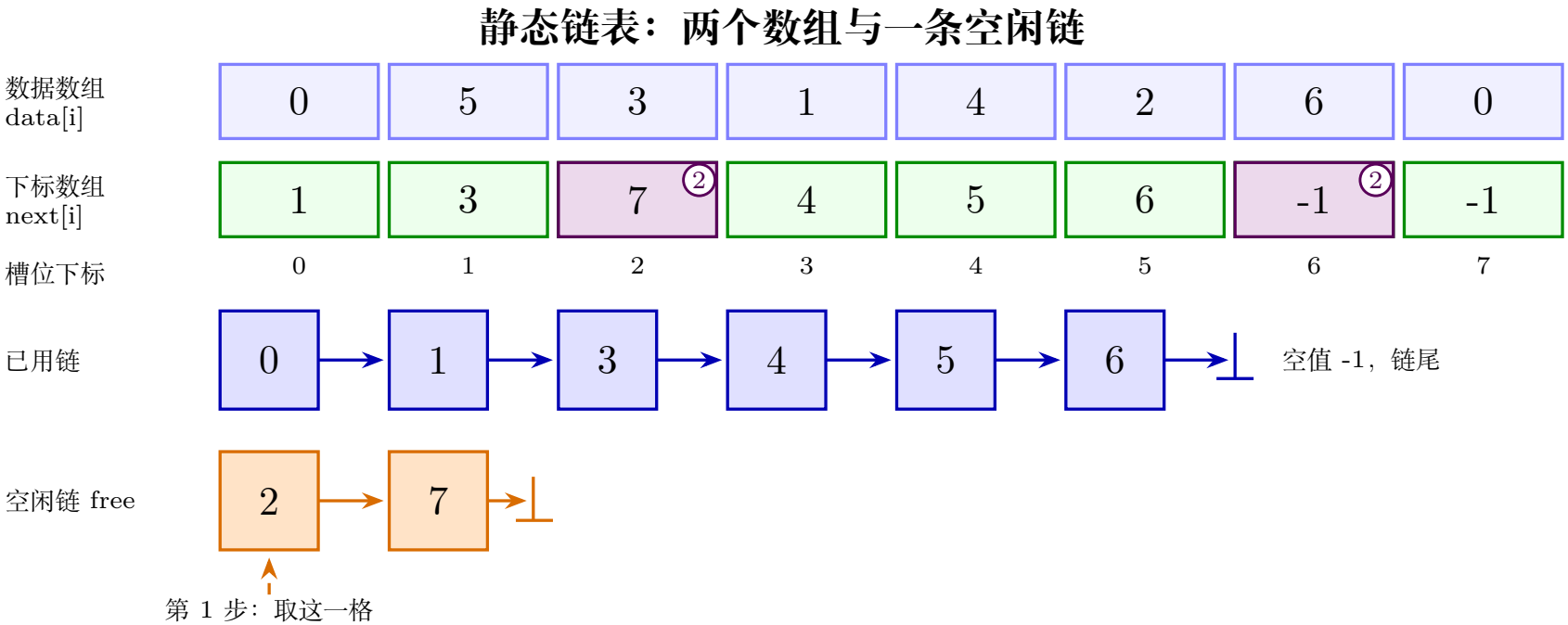
适用面四条：不支持动态分配的环境（没有 `malloc` 这类接口）、嵌入式与内存受限场景、需要重排或排序大对象（搬下标而不搬数据）、以及教学上作为理解链表与数组的桥梁；逐条的动机展开见静态链表小节的详解。

代价两条：一、**容量固定**（数组一次分配，装满之后插入失败；实测第 8 个元素就已经装不下，程序返回失败而不是扩容）；二、**空闲空间要自己管理**（空闲链、槽位的申请与释放、以及“某个槽位是否还在链上”都要靠下标约定，出错时没有任何运行时检查会替你发现）。

与两种表示的代价对照（收口第 3 讲的选型判据）：

操作	数组	单链表	静态链表
按序号访问第 k 个	一次下标运算	走链 k 步	走链 k 步（下标与指针一样只给后继）
插入 / 删除	搬移元素 $O(n)$	改指针 $O(1)$（前提是已持有前驱）	改下标 $O(1)$（前提同上）
重排 / 排序	搬移元素	改指针	改下标，数据不动
内存	一段连续区间	每节点单独申请	一次分配、容量固定、可能碎片化的空闲槽位
容量扩展	申请更大区间并复制	按需申请	不支持，只能预先给足

按序号访问与重排代价分别落在两个不同的结构上，静态链表把“重排贵”这一项换成了“改下标”，同时保留了“容量固定”这一项数组的缺点。**判据据此从“选哪一个”改成“哪一段用哪一个”**：数据一段放在数组里按序号访问，需要频繁按新次序重排的部分用下标串起来，两件事各用各的长处。



槽位 0 是哑节点，next[0] 指向第一个数据槽位 1；图中是删掉值 3 之后的状态，槽位 2 已挂回空闲链。
插一个元素分两步：第 1 步从空闲链头取一个槽位（槽位 2），把新值覆盖写进 data[2]。
第 2 步改两处 next（图中标 2 的两格）：next[6] 填 2，旧尾接上新槽位；next[2] 填 -1，新槽位成为新尾。
链的形状只写在下标数组里：插入与重排都只改下标，数据一个也没搬动（重排升序也一样）。

图 10: 静态链表的两数组与空闲链

Q 8 AI 7 详解 11 常见错误与要点（静态链表）。

1. **槽位用完不检查**：空闲链空了还继续取，会写到数组之外；申请槽位的第一步就要判 `free == -1` 并返回失败。
2. **删除时忘了挂回空闲链**：只改前驱的 `next` 而不把被删槽位挂回空闲链，那块槽位就永远拿不回来，可用容量越用越少。

3. **空闲链与数据链串混**：同一个槽位同时出现在两条链上，第一次重排或删除就会破坏结构；验收要打印空闲链的状态。
4. **重排时顺手搬了数据**：重排只需要重写下标数组，一旦写了数据数组，这一结构唯一的优势就没了。本节验收判据第一条就是为了抓这一处。
5. **容量当成可扩展**：静态链表的容量是数组长度，装满就失败；要更多容量只能换一块更大的数组并整体搬运，代价与数组扩容相同。
6. **把下标当成按序号访问**：下标数组给出的次序仍然是链表次序，按序号访问第 k 个元素还是要走链；静态链表没有把数组的 $O(1)$ 随机访问一起捡回来。

工程实战：C 项目组织与函数库

工程实战：C 项目组织与函数库

前几节的链表代码已经攒下不少：第 3 讲的建链、插入、删除与遍历，本讲的合并、拆分、逆转与检环，再加上链表上的四种排序。这些函数眼下散在各个示例脚本里，一个脚本一份主函数，改一处就要把整份脚本重编一遍。一个真正的 C 项目要把三件事的边界立清楚：对外承诺写在头文件里，实现在源文件里，测试单独放在测试目录里；再把一批函数装成一个动态链接库，让调用方不必知道实现也能用它们。

四个术语定义如下。**编译**指把 C 源文件翻译成机器码的过程。**目标文件**（后缀 `.o`）是编译的中间产物，里面是机器码与一张符号表，各符号的最终地址还没有确定。**链接**指把若干目标文件与用到的库拼在一起、确定各符号的地址、产出一个可执行文件或一个库的过程。**动态链接库**（后缀 `.so`）是一种在程序运行时才被加载的库：库的机器码不复制进可执行文件，可执行文件里只留一份引用，程序启动时由加载器把库装进内存并解析符号；同一份库文件能被多个程序共用，改库之后不必重编用它的程序。

头文件（后缀 `.h`）写一个模块对外的承诺，即类型定义与函数声明，供别的模块包含；**源文件**（后缀 `.c`）写这个模块的实现，别人不必读它。

Makefile 是一份用构建规则写成的说明文件。每条规则写清三样东西：**目标**（要产出的文件）、**依赖**（产出它需要的输入）、**命令**（产出它的动作）。make 读这份文件，比较目标与依赖的时间戳，只重做那些依赖比目标新的部分，所以改了 `sort.c` 只会重编排序那一块，不会把整个项目重来一遍。

项目里每一件东西都有它的位置，目录长这样。

- **顶层的 Makefile**：先出动态链接库，再出调用这个库的测试程序；另备 `run` 与 `clean` 两个快捷目标。
- **README.md**：写目录结构、编译与运行方法、模块与文件的对应关系。别人照着它能把项目跑起来。

- **头文件与源文件成对**：链表模块是 `list.h` 与 `list.c`，排序模块是 `sort.h` 与 `sort.c`。成对是为了让承诺与实现分开，别的模块只包含头文件。`sort.c` 要建链与打印，它包含的是 `list.h`，不是 `list.c`。
- **test 目录**：测试程序自己的源文件、自己的 Makefile 与一个测试脚本。测试程序是库的使用者，它与库之间只通过公开头文件发生关系。

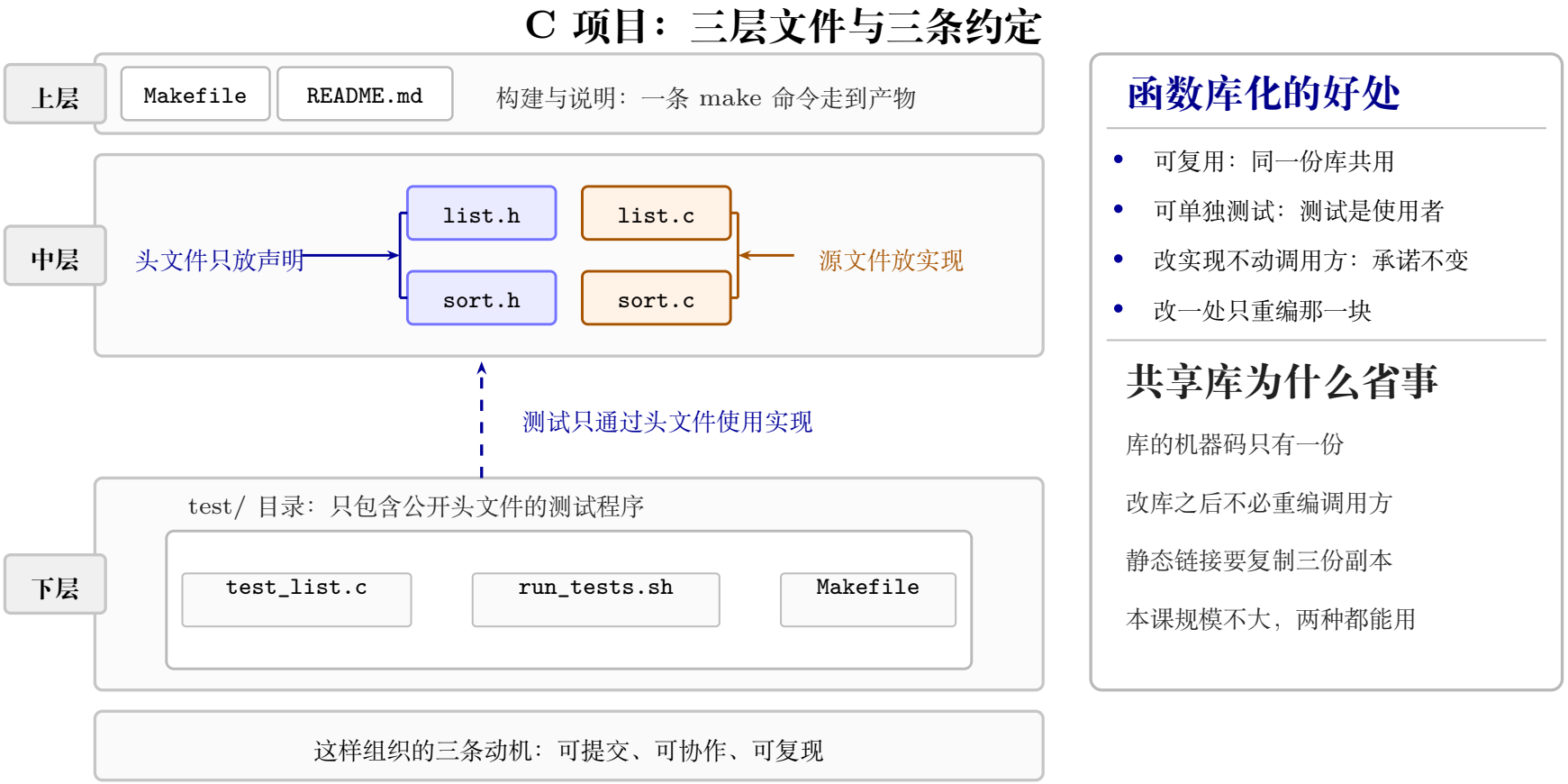


图 11：C 项目的三层文件与三条约定

这样组织的动机有三条。

1. **可提交**：作业与实验以 Git 仓库的形式提交，仓库里的目录结构本身就是评分对象的一部分，目录清楚，评阅时一眼能看出哪一块是哪一块。

2. **可协作**：承诺与实现分开之后，两个人可以同时动两个模块，只要头文件里的承诺不变，两边的改动不冲突。
3. **可复现**：一条 make 命令能从源文件走到产物，README 写清这一步怎么做，换一台机器、换一个学期都能重来。

头文件里放什么，标准是三条：类型定义（节点结构体这类被接口用到的类型）、函数声明（函数名、参数类型、返回类型）、以及必要的常量与宏。实现细节一律不放：内部用的辅助函数只声明为 `static` 写在源文件里，头文件里出现的每一个名字都是对外承诺的一部分，改它就要考虑所有调用方。

本节的产出是一个动态链接库加一个调用它的测试程序。`examples/cproject/` 是一份可运行的最小示例，实跑结果如下（本机实测）：`make` 依次给出两条编译命令、一条共享库链接命令与一条测试程序链接命令，产物是 `liblist.so` 与 `test/run_test` 两个文件；`liblist.so` 被识别为 ELF 64 位共享对象，导出七个符号（`list_from_array`、`list_print`、`list_length`、`list_free`、`list_split_half`、`list_merge_sorted`、`list_merge_sort`）；测试程序运行后三项检查全部通过，退出码为 0；`ldd` 确认可执行文件确实把 `liblist.so` 链接了进来。

两讲的链表代码怎么分层进库，本课次的建议是按“改不改结构”切三块。

模块	放什么	为什么放这里
`list.h` 与 `list.c`	节点定义、由数组建链、打印、求长度、释放，以及第 3 讲的插入与删除	这一块只依赖节点定义与指针，是其余一切模块的底座
`sort.h` 与 `sort.c`	拆分长链、合并有序链、链表上的归并排序	排序只调用底座的两个操作，自己不重新定义节点
`ops.c`（本讲新增的可选做法）	检环、相交、倒数第 k 个这类只读不改的复杂操作	这些函数不改结构，只顺着指针走，与底座分开之后接口更干净

选读：把节点改造成带类型标签的**通用节点**，即一个枚举写明这个节点装的是什么类型的数据、一个联合体存放各种类型的数据域、若干个指针域指向后继，这样同一个节点类型可以同时服务后面的树、栈与队列。它的好处是接口只写一遍，

代价是类型安全要由程序员自己守：联合体里读错成员编译器不会拦，联合体的初始化与销毁每处都要写对。本节只作选读，不要求写进作业。

库的组织与构建不进作业与验收。

Q 9 AI 8 详解 12 详解 13

课外：阅读标准库、Lisp 与 Scheme 与两种语言的分工

标准库的调度层与算法内核

标准库是一门语言随发行版一起提供的那一套基础函数与类型的集合，它是这门语言里被读得最多、也被改得最多的代码。读标准库代码对掌握一门语言见效快，收获有四条。

1. **学到语言的最佳实践与规范写法**。标准库里的命名、参数次序、错误处理方式就是这门语言的惯例，照它写，代码从第一天起就像这门语言的本地写法。
2. **理解底层实现与设计思路**。同一件事在标准库里往往有几条实现路径，把路径挑出来的依据（数据规模、元素类型、是否需要稳定排序）看明白，抽象能力跟着长。
3. **提高编程能力与代码质量**。标准库里那些反复出现的写法（提前返回、边界检查、把公共部分抽成小函数）是现成的模板，照着用能避开常见错误。
4. **掌握高级技巧与优化方法**。为了把常数压下来，标准库里用了一些单独讲要花很多篇幅的手法，读到之后，视野与工具箱同时变宽。

标准库里的代码不总是短。把一件简单的事写长的原因，正是阅读时要找的东西。**调度**指按输入的类型、长度、比较函数等条件把一次调用分派到不同实现的那一层代码，它不改变任何结果，只决定走哪条路。标准库里大量篇幅花在调度与辅助函数上，真正的算法内核只占少数几处。

实跑的参照是 `base/sort.jl` 的本机实测表（见本节末尾的详解）：公开接口与调度占了绝大多数行数，排序内核只有少数几处；算法方向上也不止一条，除了快速排序的分派方法，还有归并排序的一条支路与整数区间用的基数排序。行数随版本变化，因此阅读报告里要写清读的是哪个版本、哪一份文件、多少行。

小任务（阅读报告）：找到自己常用语言的标准库中排序算法的实现，读代码，写一份阅读报告，描述代码的每个部分如何工作。报告不少于四条内容，各条具体要求见本节末尾的详解。报告交到课程公告指定的仓库，截止时间以课程平台公告为准。

Q 10

Lisp 与 Scheme 的渊源

`nil` 与 `cons` 这两个名字来自 Lisp 与 Scheme 语言。**cons** 是这两个语言里最基本的构造子，把两个对象接成一个有序对；**nil** 是空列表的名字。一串 `cons` 嵌套起来就得到一张列表，一张列表的存储方式与链表完全一致。

渊源可以从两条线看。

- **列表到 LISP 到 lambda 演算到数学**。列表是最基础的数据结构之一，20 世纪 50 年代的 LISP 语言把它当作核心：LISP 的核心数据结构就是链表，一切复杂结构都能用链表与递归表达。LISP 的设计直接受到 **lambda 演算**（一种用函数与函数应用表达计算的形式系统，是函数式编程的理论基础）的影响，于是链表与递归把数学上的函数与数据结构直接带进了程序，LISP 因此得了“计算机科学中的数学”这个位置。
- **LISP 到 Scheme 到计算机科学**。LISP 之后，Scheme 一类语言把函数式思想推进了一步，强调代码即数据，支持**高阶函数**（把函数当作参数传入或当作结果返回的函数）与**闭包**（带着自己定义时环境的一段可调用的代码）。Scheme 与它的教材成为一些高校计算机入门课的标准教材，训练出来的是用数学的方式想程序的人。

这两条线解释了一件在本课反复出现的事：**链表与递归总是同时出现**。链表的定义本身就是递归的，一张链表要么是空表，要么是一个节点接一张链表；用递归写链表上的操作，代码与定义同形，正确性也由定义直接给出。第 3 讲 Julia 那一节里的两个类型 `Cons` 与 `Nil`，正是这套记法在 Julia 里的落地：非空链记作 `Cons`，空链记作 `Nil`，一个抽象类型下面挂两个具体类型，操作按分派落到该去的那一支。看到 Julia 侧的 `Cons` 与 `Nil` 时认出它们的来历。

AI 9

两语言问题与本课的分工

两语言问题指高性能计算里长期存在的一种两难，三条表述合起来就是它的全貌。

1. **用编译语言写代码慢**：C 这一类语言的性能好，但写起来要管的细节多，从想法到能跑的代码距离长。
2. **用解释语言跑起来慢**：一类脚本语言的写法轻快，但运行时的开销大，把算法改成能用的性能往往要再走一遍别的语言。
3. **在两种语言之间来回搬运也慢**：一部分写在快的语言里、一部分写在快的语言写的库里，拼接处的边界与数据转换既费时间又容易引入错误，调起来还要两套工具。

Julia 试图同时解决这三件事：用接近脚本语言的写法，拿到接近编译语言的运行速度，靠的是把类型信息带进编译期。两种语言各有它负责看清的东西。

- **C 负责看清内存与指针**：节点在哪里、谁申请、谁归还、空值是谁的约定、指针赋值顺序错了会断链还是成环。C 把地址与生命周期摆在台面上，链表的存储结构在 C 侧是看得见摸得着的。
- **Julia 负责看清类型与抽象**：一个结构可以由抽象类型加若干具体类型组成、空值写进字段类型、操作靠分派落到正确的实现。链表“次序由指针给出”这条结论在 Julia 侧表现为“次序由引用与类型给出”，反过来照出前一条的边界在哪里。

复杂度与性能的提示不在本节展开：链表查找做不到 $O(\log N)$，因为链表没有随机访问，连“取中点”这一步本身都是 $O(n)$，这件事第 3 讲的附录已经讲过。

三件课外的完成标准：读标准库那一件落成一份阅读报告；Lisp 与 Scheme 那一件落成一次能说清来龙去脉的复述；两语言的分工那一件落成写代码时的一个自觉，即每写一处链表操作，知道自己在看的是内存还是类型。

Q 11 详解 14 详解 15

课上练习与课后练习

课上练习：三道衔接题

本讲的课堂练习三道，每道题需要同时用到两处操作。

- **练习一：把一条长链按位置的奇偶拆成两条链**。给定一条长度任意的单链表，位置从 1 起数，把奇数位置的节点按原有的相对次序连成一条链，偶数位置的节点连成另一条链，两条链的长度最多相差 1；不新建节点，改完指针之后原链的表头指针置空。
- **练习二：交换两个相邻的节点**。给出交换第 k 个与第 $k+1$ 个节点的步骤，并说明相邻情形与不相邻情形相比，在哪一步上不同。
- **练习三：在三路快速排序的拆分上，补出把三条链接回去的一步**。按枢轴把节点分成小于、等于、大于三条链之后，三条链的先后已由大小关系定死，要补的是把它们接回成一条链的那一步；空链要跳过，接回之后不得丢节点、不得重复。

三道题的共同验收判据四条：**写出操作步骤**，写的是改哪几条指针、顺序如何，不停在“交换两个节点”这样的结论上；**能实跑验证**，改完指针之后链上节点数不变、无断链、无成环；**边界情形有处理**，空链、单节点链、拆出空链、头节点参与交换、位置越界、三条链里有空链这些都是要各自想一遍的地方；**不新建节点**，三道题都只改指针。

三道题各自的前置知识都能指回已讲过的内容：练习一用第 3 讲的基本操作（节点与改指针）与按位置计数的游标遍历；练习二用第 3 讲的前驱关系与本讲复杂操作一节的相邻与不相邻两种情形；练习三用第 2 讲的快速排序与分治、以及本讲链表排序一节的按枢轴分区。

Q 12 AI 10 详解 16

课后练习：四道

课后练习四道，练的是本讲讲过的排序、双指针与哨兵类操作，做完教师课上讲评。

- **练习一：在单链表上实现冒泡排序与归并排序。** 两种排序各写一个函数，可独立编译运行；每种排序给一组测试输出，输出里能看出输入与结果；冒泡排序交换节点或交换数据域两种写法都要说明代价差在哪里；归并排序由拆分长链与合并有序链两个基本操作组合而成，这两个函数单独可调用。测试输入至少覆盖已有序、逆序、含大量重复值三种，每种给出比较次数与改指针次数；含大量重复值的那种要说明三路分区为什么在这类输入上比二路分区省。语言 C 与 Julia 用一种即可。
- **练习二：检环与检相交。** 检环：判断链表中是否有环，若有则找出环的起始节点；检相交：判断两条给定的链是否相交，若相交则找出交点。测试数据自己构造，带环与不带环、相交与不相交都要有，说明里写清环长、环起点与交点位置，并能解释这份数据为什么验证的就是要验证的那件事。
- **练习三：返回倒数第 k 个节点。** 写出双指针的间距不变式；$k = 1$、k 等于链长、k 大于链长三种输入各给出结果，k 大于链长时返回“不存在”。
- **练习四：读一份链表参考实现，改写自己的链表示例。** 课程随题分发一份带头节点的链表参考实现，读它；为自己的链表示例补上链表结构（节点定义，以及由数组建链、打印、求长度、释放四个基本函数），并参考其中的**哑节点**（哨兵：一个不存放有效数据的头节点，它的存在把所有节点的前驱都变成“一定存在”，因此不再需要单独处理删头这一支）方法，把第 3 讲写过的创建、插入、删除、打印四个函数改写一遍；改写前后各给一遍输出，说明改写省掉了哪一类分支处理。

AI 11 本讲的作业在下一节《作业》完整布置。

作业：任务与步骤的链表与文件 IO

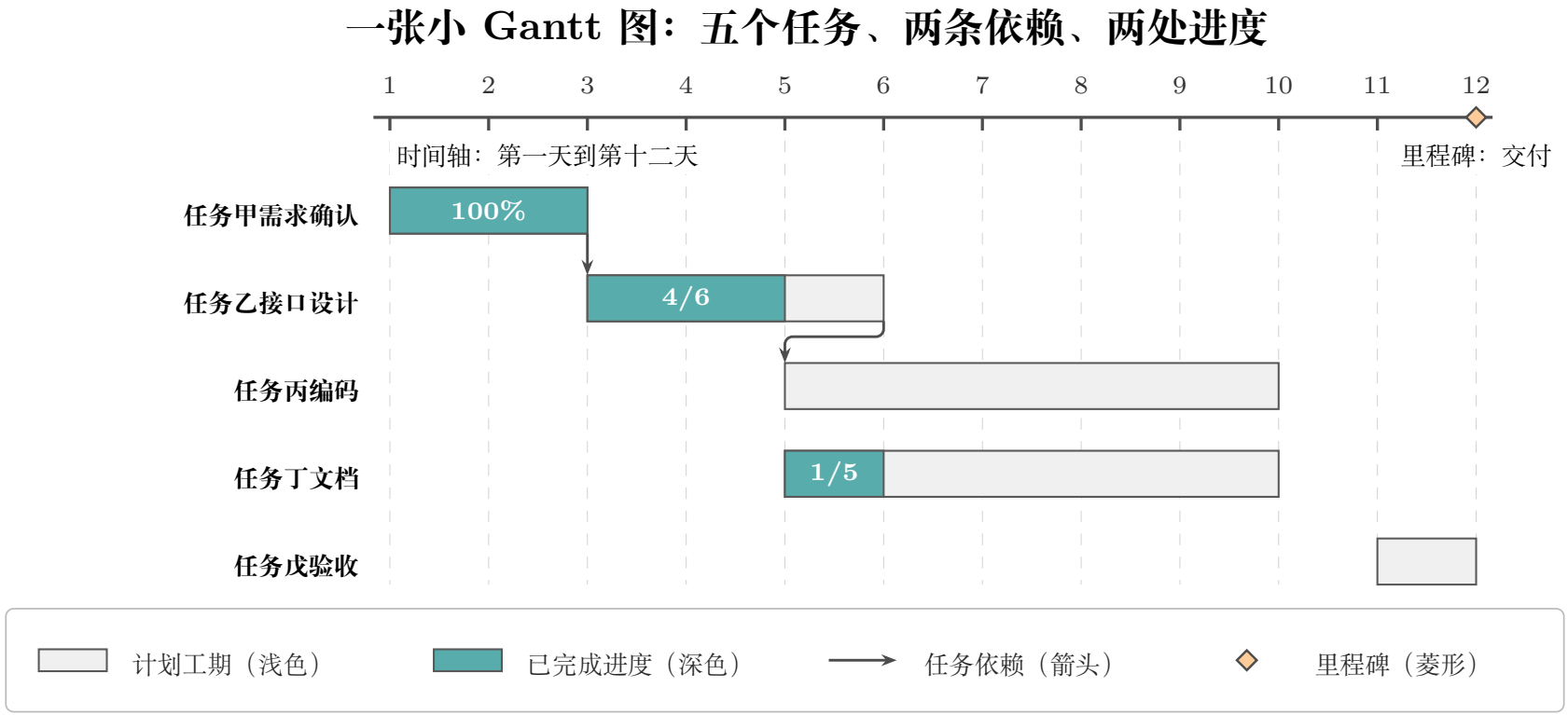
Gantt 图是什么：任务、时间轴、条形、依赖与进度

Gantt 图（甘特图）是一种横向条形图：用一根从左到右推进的时间轴承载若干任务，每个任务占一根横条，横条的左右两端是任务的开始与结束时刻，横条上按完成比例填充的部分给出进度，任务之间的先后约束用连接条或箭头画出。七个术语在这里给出完整定义（术语对照表中另含总名 Gantt 图与派生量工期两项）。

- **任务**（task）：一件有明确目标、可以独立安排与跟踪的工作单元。在 Gantt 图里它占一根横条；在数据上它通常带一个标识、一个名称与一段起止时间。
- **步骤**（step）：任务内部按顺序执行的一个动作。一个任务由一串有序的步骤组成，步骤之间的先后是执行次序，不是依赖里那种可以并行、可以交叉的约束。
- **时间轴**（timeline）：图下方从左到右推进的时间刻度。横条的左右两端直接落在刻度上，读图时先看刻度再看横条。
- **条形**（bar）：图中代表一个任务的那根横条。它的左端对应该任务的开始时刻，右端对应结束时刻，长度对应该任务的**工期**，即起止时刻之间的时间长度。
- **进度**（progress）：横条上按完成比例填充的那一部分，回答这项任务做到哪一步了。填充比例与横条长度是两个不同的量：长度是计划的时间跨度，填充是实际推进的程度。
- **依赖**（dependency）：两个任务之间的先后约束，在图上用连接条或箭头表示，含义是前一个任务不结束，后一个任务不能开始。
- 工期与进度之外，图上还可能出现**里程碑**（milestone）：一个只有时刻、没有时长的检查点，通常画成一个菱形或一根零宽度的标记，用来说明某件事必须在某个时刻之前完成。

一张图看四样东西：纵轴是任务清单，横轴是时间轴，每条横条是一个任务的时间占用，横条上的填充与任务之间的连接条给出进度与依赖。**读图的三条规则**：

1. **看位置与长度定时间**：横条的长度就是工期，横条的左端就是开始时刻，右端就是结束时刻。
2. **看并排定并行**：同一时间区间上并排的多根横条，表示这几件事可以同时做。
3. **看连接条定先后**：连接条指出的次序就是先后约束，箭头从哪个任务的结束指向哪个任务的开始，就表示前者不完成后者不能动。



横条长度即工期，左端是开始时刻、右端是结束时刻；浅色为剩余计划段，深色为已完成进度。

连接条由前者末尾指向后者开头，含义是前者不结束后者不能开始：甲到乙同刻相接，乙到丙折行跨过一行。

丙与丁的时间区间完全重合，图上只表示两件事可以同时做，并不说明能否由同一个执行者做。

图 12: 小 Gantt 图：任务、依赖与进度

本课的大作业是一个**任务甘特图项目管理平台**，它是理论课的课程项目，以树组织任务的层级，以有向无环图表达任务之间的依赖，实现依赖顺序的排列与进度计算，最终生成甘特图视图；它按检查点分阶段推进。本节先只取这张图里两样最小的信息：**一个任务由一串有序的步骤组成**，以及**一个步骤可以带一段起止时间**。

Q 13 AI 12 详解 17

Gantt 图的用处、优点与局限

Gantt 图能回答的问题可以归成六类。

1. **哪些任务要做**：纵轴那一列就是答案，图上没有的任务不在计划里。
2. **各在什么时候做**：每根横条的位置就是各自的开始与结束时刻。
3. **哪些任务可以并行**：时间区间重合、并排出现的横条就是可以同时做的那几件。
4. **整件事还要多久**：最后一根横条的右端离现在有多远，就是计划上的剩余时间。
5. **哪些任务卡住了后续**：被连接条指着的那几个任务，只要它们没结束，后面的就动不了。
6. **眼下进度落后多少**：把每根横条的填充位置与时间轴上“今天”这一根竖线对照，落后与提前一眼可见。

项目管理里用它做四件事：**排计划**，把任务摊到时间轴上，先定次序再定时刻；**对齐分工**，同一张图上看清谁在什么时候忙、谁在什么时候空；**跟踪进度**，横条的填充随实际推进更新；**向上汇报**，一张图代替一段文字说明。

它的四条优点：

1. **直观**：时间占用与先后关系变成位置与长度，不依赖文字描述就能读懂，读图的门槛几乎为零。
2. **沟通成本低**：同一张图可以给不同角色看，讨论与修改围绕同一份图形进行，不必各自在脑子里重建一份计划。
3. **与日历对齐**：横条直接落在日期刻度上，排期与周次、节假日、课程安排这类真实的时间边界自然对上。
4. **计划与实际可以放在一处对照**：计划条与实际条并排画，落后与提前在图上是一条缝隙的宽窄，不需要再去比对两份表。

每条局限都决定了它不适合做什么。

1. **难以表达复杂依赖**：交叉约束、多对多的先后、只有满足某个条件才执行的分支，在条形图上只能靠附加标注，图本身表达不了。

2. **难以表达资源约束**：同一个执行者在同一时间只能做一件事这类限制，图上只能看到时间重叠，看不到资源冲突，也看不到怎么把负载摊平。
3. **变更时维护成本高**：任务一改，横条位置、连接条与进度都要跟着改；任务一多，图变密集，逐条读图变慢。
4. **表示计划不表示工作量**：横条的长度是时间跨度，不等于工作量，也不说明这项任务有多难、需要几个人。

Q 14 AI 13 详解 18

时间相洽性检查：本课的判定规则

相洽指同一串步骤内部在时间上说得通，不存在自相矛盾的时间。它至少包含两层：一是单步自身开始与结束的关系，二是同一串步骤内两步之间的关系。两层之上还有一条前提，即哪些步骤参与检查、哪些不参与。

本课的时间约定：一个步骤占用的时刻记成一个**半开区间** `[start, end)`，其中 `end` 是这一步不占用的时刻。也就是说，这一步占用的时刻是 start 到 end 之间不含 end 的那些时刻。取半开区间有一个直接好处：前一步的结束时刻本身不占用时间，后一步可以从这个时刻开始，两步既不重叠也不留缝。这条约定由本课定。

三条可执行条件。

1. **自身相洽**：起止都存在时要求 `start ≤ end`。`start` 晚于 `end` 为不相洽，须报错。这条条件管的是单步自身，与别的步骤无关。
2. **零时长**：`start` 等于 `end` 表示零时长的步骤，它占用的时刻为空，是一个**空区间**。本课按空区间处理并允许存在，例如只需记录一个时刻的检查点。空区间与任何步骤都不冲突。
3. **两步重叠**：把两步记为 `[s_i, e_i)` 与 `[s_j, e_j)`，两步重叠当且仅当 **`s_i < e_j` 且 `s_j < e_i`**。重叠即相洽性冲突，重叠区间是 `[max(s_i, s_j), min(e_i, e_j))`。

第 3 条有一处前提必须写明：它只在两步都已排期、且两步的区间都非空时成立。空区间里一个时刻都没有，它与任何步骤都不冲突；若把空区间直接套进重叠公式，位于另一个步骤区间内部的零时长步骤会被判成重叠，这是一个真会发生的错判。

未排期的步骤不参与检查：起止时间缺省的步骤跳过重叠判定，既不报冲突也不报错；只给一端时间的情形按未排期处理，同样跳过。

必须区分的六种情形。

情形	条件	判定	重叠区间
完全分离	`e_i ≤ s_j` 或 `e_j ≤ s_i`	不冲突	无
首尾相接	`e_i = s_j` 或 `e_j = s_i`	不冲突	无
部分交叉	`s_i < s_j < e_i < e_j` 或对称	冲突	`[s_j, e_i)` 或对称
一个包含另一个	`s_i ≤ s_j` 且 `e_j ≤ e_i` 或对称	冲突	`[s_j, e_j)` 或对称
零时长	`s_i = e_i`	不冲突，按空区间跳过	无
未排期	起止缺省或只给一端	跳过检查	无

首尾相接为什么不算冲突：判定见上表，理由即“本课的时间约定”里半开区间的那条好处。

两种检查做法。

1. **两两比较**：遍历链表，对每一对步骤作一次重叠判定。时间代价 $O(n^2)$，额外空间 $O(1)$；它能把**全部**冲突对与重叠区间都列出来。
2. **先按起始时刻排序，再比较相邻两项**：排序复用本讲的链表排序，时间代价 $O(n \log n)$；未排期与零时长的步骤在排序前挑出来单独放。它能在更短的时间里回答“有没有冲突”，并定位到相邻位置上的一处冲突，但它**给不出全部冲突对**：一步可能与好几步重叠，而其中有些步骤在排序后并不与它相邻。

Q 15 AI 14 详解 19

作业

截止时间：以课程平台公告为准。语言用 C 与 Julia 之一即可。

编程任务

1. **用链表实现任务**。一个任务由一串有序的步骤组成，这串步骤用单链表存放。每个步骤要有一个能唯一认出它的标识（报冲突时用它指出是哪两步）。实现三种操作：在指定步骤之后插入一个步骤、删除一个步骤、从头到尾遍历全部步骤；每次操作之后链上步骤数与预期一致。步骤在链上的先后由指针给出，不得另用一个数组或下标来记顺序。
2. **步骤带可选的起止时间，给了就做时间相洽性检查**。每个步骤可以带一段起止时间，也可以不带；不带（或只给一端）一律算**尚未排期**。检查按上一节的三条可执行条件执行：单步 `start ≤ end`，违反报错；已排期的两步区间重叠即为冲突，程序要输出**全部**冲突的步骤对与每一对的重叠区间（输出口径见上一节两两比较与排序相邻比较的实测对照）。
3. **设计文件 IO**。把一个任务连同它的步骤链写到一份文本文件，再从这份文件读回同一个结构。本项只交**设计**，不交实现：设计文档要写到别人照着它就能把实现写出来的程度，逐条回答八个问题：格式选型与理由；写哪些字段、顺序是否固定；尚未排期在文件里怎么表示、怎么与“时间为零”区分；名称里出现分隔符怎么办；写到一半失败怎么办；读回时格式不对在哪一步报错；相洽性检查在写前做、读后做还是两处都做；格式要不要版本标记、以后加字段怎么办。
4. **交付四件**：源码（第 1、2 两项的实现）；测试用例（上一节六种相洽情形各有一组输入与预期输出）；设计文档加一份按你的格式写成的**样例文件**（纯文本）；README（结构说明与运行方法，时间精度与非法值处理写在这里）。

验收看什么：源码在 `clang -Wall -Wextra -g` 下零告警、可独立运行；冲突输出是全部冲突对而不只是“有冲突”；尚未排期、零时长、非法时间在数据和文件里是三件不同的事，读回来不会互相转化；设计文档无歧义（任取一行按文档只有一种拆法）、错误清单覆盖空文件、缺行、字段缺失、时间格式非法这些情形；样例文件与文档一致。

AI 15 Q 16 Q 17